PANDUAN PRAKTIKUM

Container Security dengan Docker & Trivy

Mata Kuliah DevSecOps – Pertemuan 9

S1 Teknik Informatika | STT Nurul Fikri

Disusun oleh:

April Rustianto, S.Komp., M.T., CCIE

Digunakan untuk keperluan akademik internal STT Nurul Fikri

Informasi Praktikum

Mata Kuliah	DevSecOps
Pertemuan	9 – Container Security dengan Docker & Trivy
Durasi	± 2 × 50 Menit (2 SKS Praktikum)
Prasyarat	Sudah menginstal Docker Desktop / Docker Engine
Platform	Linux / macOS / Windows (WSL2 direkomendasikan)
Dosen	April Rustianto, S.Komp., M.T., CCIE

Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa mampu:

- Menjalankan dan menginspeksi container Docker secara mandiri.
- Menggunakan Trivy untuk melakukan scanning kerentanan pada container image.
- Membuat Dockerfile yang aman dengan menerapkan best practices keamanan.
- Mengintegrasikan Trivy ke dalam pipeline CI/CD menggunakan GitHub Actions.

Lab 1 – Docker Fundamentals

Durasi	± 30 Menit
Tujuan	Menjalankan dan memahami dasar-dasar container Docker
Prasyarat	Docker sudah terinstal dan berjalan (jalankan: docker --version)

1.1 Verifikasi Instalasi Docker

Langkah 1: Cek versi Docker yang terinstal

i INFO Perintah ini memastikan Docker sudah terinstal dengan benar dan service-nya berjalan.

```
$ docker --version
$ docker info
```

Output yang diharapkan:

```
Docker version 24.0.5, build ced0996
...
Server Version: 24.0.5
Storage Driver: overlay2
```

Langkah 2: Jalankan container "Hello World"

```
$ docker run hello-world
```

Output sukses akan muncul pesan sambutan dari Docker:

```
Hello from Docker!
This message shows that your installation appears to be working
correctly.

1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
3. The Docker daemon created a new container from that image.
4. The Docker daemon streamed that output to the Docker client.
```

1.2 Mengelola Container dan Image

Langkah 1: Jalankan container Ubuntu secara interaktif

i INFO Flag -it artinya interactive + tty, memungkinkan kamu berinteraksi dengan shell di dalam container. Flag --rm memastikan container otomatis dihapus setelah keluar.

```
$ docker run -it --rm ubuntu:22.04 /bin/bash
```

Kamu akan masuk ke dalam shell container. Jalankan perintah-perintah berikut di dalam container:

```
# Di dalam container:
root@abc123:/# cat /etc/os-release
root@abc123:/# uname -a
root@abc123:/# ps aux
root@abc123:/# exit
```

Langkah 2: Jalankan container Nginx sebagai web server

i INFO Flag -d menjalankan container di background (detached). Flag -p memetakan port host:container.

```
$ docker run -d --name my-nginx -p 8080:80 nginx:latest
```

Verifikasi container berjalan:

```
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED
STATUS         PORTS                    NAMES
```

```
a1b2c3d4e5f6   nginx:latest   "/docker-entrypoint.…"   5 seconds ago
Up 4 seconds   0.0.0.0:8080->80/tcp   my-nginx
```

Akses Nginx di browser: http://localhost:8080 — kamu akan melihat halaman default Nginx.

Langkah 3: Inspeksi container yang berjalan

```
# Melihat log container
$ docker logs my-nginx

# Inspeksi detail container (network, mounts, env, dll)
$ docker inspect my-nginx

# Melihat penggunaan resource (CPU, Memory)
$ docker stats my-nginx --no-stream
```

Langkah 4: Masuk ke dalam container yang sedang berjalan

```
$ docker exec -it my-nginx /bin/bash
```

Di dalam container, cek konfigurasi Nginx:

```
# Di dalam container my-nginx:
root@a1b2c3d4:/# cat /etc/nginx/nginx.conf
root@a1b2c3d4:/# ls /usr/share/nginx/html/
root@a1b2c3d4:/# exit
```

Langkah 5: Hentikan dan bersihkan container

```
$ docker stop my-nginx
$ docker rm my-nginx

# Melihat semua image yang ada
$ docker images

# (Opsional) Hapus image yang sudah tidak dipakai
$ docker image prune -f
```

> ✓ **CATATAN** Lab 1 selesai! Kamu sudah berhasil menjalankan container, mengeksplorasi isinya, dan mengelola lifecycle container.

Lab 2 – Container Vulnerability Scanning dengan Trivy

Durasi	± 45 Menit
Tujuan	Menginstal Trivy dan melakukan scanning kerentanan pada container image
Tools	Trivy (trivy.dev), Docker

2.1 Instalasi Trivy

Langkah 1: **Instal Trivy (pilih sesuai OS)**

Linux (Ubuntu/Debian)

```
$ sudo apt-get install wget apt-transport-https gnupg lsb-release -y
$ wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key |
\
    sudo apt-key add -
$ echo deb https://aquasecurity.github.io/trivy-repo/deb \
    $(lsb_release -sc) main | \
    sudo tee -a /etc/apt/sources.list.d/trivy.list
$ sudo apt-get update
$ sudo apt-get install trivy -y
```

macOS (Homebrew)

```
$ brew install trivy
```

Windows (Scoop)

```
$ scoop install trivy
```

Atau menggunakan Docker (semua OS)

> **i INFO** Cara ini tidak memerlukan instalasi Trivy secara langsung. Trivy dijalankan melalui Docker.

```
$ docker run --rm -v /var/run/docker.sock:/var/run/docker.sock \
    aquasec/trivy:latest image nginx:latest
```

Langkah 2: **Verifikasi instalasi Trivy**

```
$ trivy --version
Version: 0.48.3
Vulnerability DB: ...
```

Update database vulnerability ke versi terbaru:

```
$ trivy image --download-db-only
```

2.2 Scanning Image yang Vulnerable

Langkah 1: **Scan image Python versi lama (sengaja vulnerable)**

> **⚠ PERHATIAN** Image python:3.4-alpine sengaja dipilih karena versi lama mengandung banyak CVE. JANGAN gunakan image ini di production!

```
$ trivy image python:3.4-alpine
```

Trivy akan menampilkan daftar CVE yang ditemukan. Perhatikan kolomnya:

```
python:3.4-alpine (alpine 3.6.5)

Total: 37 (UNKNOWN: 0, LOW: 4, MEDIUM: 13, HIGH: 15, CRITICAL: 5)
```

```
┌──────────────────┬────────────────┬──────────┬──────────────────────┬──────────────┐
│      Library     │ Vulnerability  │ Severity │  Installed Version   │ Fixed Version │
├──────────────────┼────────────────┼──────────┼──────────────────────┼──────────────┤
│ musl             │ CVE-2019-14697 │ CRITICAL │ 1.1.16-r14           │ 1.1.22-r3    │
│ sqlite-libs      │ CVE-2019-9936  │ HIGH     │ 3.21.0-r1            │ 3.28.0-r0    │
│ ...              │ ...            │ ...      │ ...                  │ ...          │
└──────────────────┴────────────────┴──────────┴──────────────────────┴──────────────┘
```

Langkah 2: Filter berdasarkan severity

Fokus hanya pada vulnerability CRITICAL dan HIGH:

```
$ trivy image --severity CRITICAL,HIGH python:3.4-alpine
```

Hanya tampilkan vulnerability yang sudah ada fixnya:

```
$ trivy image --severity CRITICAL,HIGH --ignore-unfixed python:3.4-alpine
```

Langkah 3: Ekspor hasil scan ke format JSON

> **i INFO** Format JSON berguna untuk integrasi dengan tools lain atau pembuatan laporan otomatis.

```
$ trivy image --format json --output scan-result.json python:3.4-alpine
$ cat scan-result.json | python3 -m json.tool | head -50
```

Langkah 4: Ekspor hasil scan ke format SARIF

> **i INFO** SARIF (Static Analysis Results Interchange Format) adalah format standar yang dapat dibaca GitHub Security tab.

```
$ trivy image --format sarif --output scan-result.sarif python:3.4-alpine
```

Langkah 5: Bandingkan dengan image yang aman (Python terbaru)

```
$ trivy image --severity CRITICAL,HIGH python:3.12-alpine
```

Perhatikan perbedaan jumlah CVE antara python:3.4-alpine vs python:3.12-alpine. Image terbaru selalu lebih aman.

2.3 Scanning Filesystem Lokal

Langkah 1: Buat direktori proyek sederhana

```
$ mkdir ~/lab-trivy && cd ~/lab-trivy
$ cat > requirements.txt << EOF
django==2.0.0
requests==2.18.0
pillow==5.0.0
EOF
```

> **⚠ PERHATIAN** Package-package di atas sengaja menggunakan versi lama yang memiliki CVE.

Langkah 2: Scan filesystem untuk dependency vulnerability

```
$ trivy fs --severity CRITICAL,HIGH ~/lab-trivy/

requirements.txt (pip)

Total: 8 (HIGH: 5, CRITICAL: 3)

┌──────────────┬────────────────┬──────────┬──────────────────┬─────────┐
```

```
┌──────────────┬────────────────┬──────────┬──────────────────────┬────────
Version┐
├──────────────┼────────────────┼──────────┼──────────────────────┼────────
────────┤
│ django       │ CVE-2019-14234 │ CRITICAL │ 2.0.0                │ 2.0.8
│
│ pillow       │ CVE-2020-5313  │ HIGH     │ 5.0.0                │ 7.1.0
│
└──────────────┴────────────────┴──────────┴──────────────────────┴────────
────────┘
```

Langkah 3: Fix vulnerability dengan update version

```
$ cat > requirements.txt << EOF
django==4.2.8
requests==2.31.0
pillow==10.1.0
EOF

# Scan ulang setelah update
$ trivy fs --severity CRITICAL,HIGH ~/lab-trivy/
```

Seharusnya tidak ada lagi CVE CRITICAL atau HIGH setelah update.

> ✅ **CATATAN** Lab 2 selesai! Kamu sudah bisa menggunakan Trivy untuk scanning image dan filesystem, serta memahami cara membaca laporan CVE.

Lab 3 – Secure Dockerfile Best Practices

Durasi	± 45 Menit
Tujuan	Membuat Dockerfile yang aman dengan menerapkan prinsip least privilege dan minimal base image
Tools	Docker, Trivy, text editor

3.1 Dockerfile Tidak Aman vs Aman

Langkah 1: Buat direktori proyek

```
$ mkdir ~/lab-dockerfile && cd ~/lab-dockerfile
```

Langkah 2: Buat Dockerfile TIDAK AMAN (sebagai perbandingan)

> 🔴 **PENTING** Dockerfile ini sengaja dibuat tidak aman. JANGAN gunakan pola ini di production!

```
$ cat > Dockerfile.insecure << 'EOF'
# ❌ BAD: Menggunakan base image besar dengan banyak package tidak
diperlukan
FROM ubuntu:18.04

# ❌ BAD: Menjalankan apt-get tanpa membersihkan cache
RUN apt-get update
RUN apt-get install -y python3 python3-pip curl wget

# ❌ BAD: Meng-copy seluruh direktori (termasuk file sensitif)
COPY . /app

# ❌ BAD: Tidak set WORKDIR
RUN cd /app && pip3 install -r requirements.txt

# ❌ BAD: Berjalan sebagai root
# (tidak ada USER directive)

# ❌ BAD: Menyimpan credential di ENV
ENV DB_PASSWORD=supersecret123
ENV API_KEY=abc123xyz

EXPOSE 8080
CMD ["python3", "/app/app.py"]
EOF
```

Langkah 3: Buat aplikasi Python sederhana

```
$ cat > app.py << 'EOF'
from http.server import HTTPServer, SimpleHTTPRequestHandler
import os

class Handler(SimpleHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.end_headers()
        self.wfile.write(b"Hello, Secure World!")

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 8080))
    server = HTTPServer(("0.0.0.0", port), Handler)
    print(f"Server running on port {port}")
    server.serve_forever()
EOF

$ cat > requirements.txt << 'EOF'
```

```
flask==3.0.0
EOF
```

Langkah 4: Build image tidak aman dan scan

```
$ docker build -f Dockerfile.insecure -t app-insecure:v1 .
$ trivy image --severity CRITICAL,HIGH app-insecure:v1
```

Catat jumlah CVE yang ditemukan pada image tidak aman ini.

Langkah 5: Buat Dockerfile AMAN dengan best practices

> ✅ **CATATAN** Dockerfile ini menerapkan: multi-stage build, non-root user, minimal base image, dan menghindari kebocoran credential.

```
$ cat > Dockerfile.secure << 'EOF'
# ✅ STAGE 1: Build stage - instal dependency
FROM python:3.12-alpine AS builder

# ✅ GOOD: Set workdir dari awal
WORKDIR /build

# ✅ GOOD: Copy hanya file yang diperlukan, bukan seluruh direktori
COPY requirements.txt .

# ✅ GOOD: Install dependency dan bersihkan cache dalam satu RUN
RUN pip install --no-cache-dir --user -r requirements.txt

# ✅ STAGE 2: Runtime stage - image final yang minimal
FROM python:3.12-alpine AS runtime

# ✅ GOOD: Buat non-root user
RUN addgroup -g 1001 -S appgroup && \
    adduser -u 1001 -S appuser -G appgroup

# ✅ GOOD: Set workdir
WORKDIR /app

# ✅ GOOD: Copy dependency dari build stage
COPY --from=builder /root/.local /home/appuser/.local

# ✅ GOOD: Copy hanya source code yang diperlukan
COPY --chown=appuser:appgroup app.py .

# ✅ GOOD: Credential dari environment (injected saat runtime, bukan
hardcode)
# ENV DB_PASSWORD - JANGAN PERNAH hardcode di sini!

# ✅ GOOD: Jalankan sebagai non-root user
USER appuser

# ✅ GOOD: Expose port yang dipakai
EXPOSE 8080

# ✅ GOOD: Gunakan ENTRYPOINT + CMD untuk fleksibilitas
ENTRYPOINT ["python3"]
CMD ["app.py"]
EOF
```

Langkah 6: Build image aman dan scan

```
$ docker build -f Dockerfile.secure -t app-secure:v1 .
$ trivy image --severity CRITICAL,HIGH app-secure:v1
```

Bandingkan jumlah CVE antara app-insecure:v1 dan app-secure:v1. Image aman seharusnya memiliki jauh lebih sedikit CVE.

Langkah 7: Verifikasi container berjalan sebagai non-root

```
# Jalankan image aman
$ docker run -d --name app-secure -p 8080:8080 app-secure:v1

# Cek user yang menjalankan proses
$ docker exec app-secure whoami
$ docker exec app-secure id
appuser
uid=1001(appuser) gid=1001(appgroup) groups=1001(appgroup)
```

Proses berjalan sebagai appuser (uid=1001), bukan root (uid=0). Ini adalah praktik keamanan yang benar.

```
$ docker stop app-secure && docker rm app-secure
```

Langkah 8: Bandingkan ukuran image

```
$ docker images | grep -E "app-insecure|app-secure"
REPOSITORY     TAG    IMAGE ID       CREATED          SIZE
app-secure     v1     abc123def456   2 minutes ago    58MB
app-insecure   v1     789xyz012345   5 minutes ago    412MB
```

> **i INFO** Image aman jauh lebih kecil (58MB vs 412MB). Image yang lebih kecil = attack surface lebih kecil = lebih aman!

3.2 Menggunakan .dockerignore

Langkah 1: Buat file .dockerignore

> **i INFO** File .dockerignore mencegah file sensitif masuk ke dalam Docker build context.

```
$ cat > .dockerignore << 'EOF'
# Credential dan config rahasia
.env
*.env
.env.*

# Git
.git
.gitignore

# Cache Python
__pycache__/
*.pyc
*.pyo
.pytest_cache/

# IDE
.vscode/
.idea/

# File test
tests/
*.test.py

# Dokumentasi
docs/
README.md
Dockerfile*
EOF
```

Verifikasi .dockerignore bekerja dengan membuat file sensitif:

```
$ echo "DB_PASSWORD=rahasia123" > .env
$ docker build -f Dockerfile.secure -t app-secure:v2 .

# Verifikasi .env TIDAK ada di dalam image
$ docker run --rm app-secure:v2 ls -la /app/
```

File .env tidak seharusnya muncul di dalam container.

> ✅ **CATATAN** Lab 3 selesai! Kamu sudah berhasil membuat Dockerfile aman dengan multi-stage build, non-root user, minimal base image, dan .dockerignore.

Lab 4 – Integrasi Trivy di CI/CD Pipeline (GitHub Actions)

Durasi	± 30 Menit
Tujuan	Mengintegrasikan Trivy ke dalam pipeline CI/CD untuk scanning otomatis
Prasyarat	Akun GitHub (gratis), Git terinstal
Tools	GitHub Actions, Trivy, Docker

> **i INFO** CI/CD (Continuous Integration/Continuous Deployment) memastikan bahwa setiap perubahan kode secara otomatis di-scan keamanannya sebelum masuk ke production.

4.1 Setup Repository GitHub

Langkah 1: Inisialisasi Git repository

```
$ cd ~/lab-dockerfile
$ git init
$ git add app.py requirements.txt Dockerfile.secure .dockerignore
$ git commit -m "Initial commit: secure dockerized app"
```

Langkah 2: Buat repository di GitHub dan push

Buka https://github.com/new dan buat repository baru bernama "devsecops-lab-p9"

```
$ git remote add origin https://github.com/<username>/devsecops-lab-
p9.git
$ git branch -M main
$ git push -u origin main
```

4.2 Membuat GitHub Actions Workflow

Langkah 1: Buat direktori workflow GitHub Actions

```
$ mkdir -p .github/workflows
$ cat > .github/workflows/security-scan.yml << 'EOF'
name: Container Security Scan

# Trigger: scan otomatis pada push dan pull request
on:
  push:
    branches: [ "main", "develop" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build-and-scan:
    name: Build & Security Scan
    runs-on: ubuntu-latest

    steps:
      # 1. Checkout source code
      - name: Checkout repository
        uses: actions/checkout@v4

      # 2. Build Docker image
      - name: Build Docker image
        run: |
          docker build -f Dockerfile.secure \
            -t my-app:${{ github.sha }} .

      # 3. Jalankan Trivy untuk scan image
      - name: Run Trivy vulnerability scanner
```

```
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: "my-app:${{ github.sha }}"
          format: "sarif"
          output: "trivy-results.sarif"
          severity: "CRITICAL,HIGH"
          exit-code: "1"  # Gagalkan pipeline jika ada CVE kritis

      # 4. Upload hasil SARIF ke GitHub Security tab
      - name: Upload Trivy scan results to GitHub Security
        uses: github/codeql-action/upload-sarif@v3
        if: always()  # Upload meskipun step sebelumnya gagal
        with:
          sarif_file: "trivy-results.sarif"
          category: "container-scanning"
EOF
```

Langkah 2: Tambahkan workflow scan untuk filesystem (SCA)

i INFO SCA (Software Composition Analysis) memastikan dependency/library yang dipakai tidak memiliki CVE.

```
$ cat > .github/workflows/dependency-scan.yml << 'EOF'
name: Dependency Security Scan

on:
  push:
    branches: [ "main" ]
  schedule:
    # Scan otomatis setiap hari pukul 00:00 UTC
    - cron: "0 0 * * *"

jobs:
  dependency-scan:
    name: Scan Dependencies
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Run Trivy filesystem scan
        uses: aquasecurity/trivy-action@master
        with:
          scan-type: "fs"
          scan-ref: "."
          format: "sarif"
          output: "fs-trivy-results.sarif"
          severity: "CRITICAL,HIGH"

      - name: Upload Trivy fs scan results
        uses: github/codeql-action/upload-sarif@v3
        if: always()
        with:
          sarif_file: "fs-trivy-results.sarif"
          category: "fs-scanning"
EOF
```

Langkah 3: Commit dan push workflow

```
$ git add .github/
$ git commit -m "ci: add Trivy security scanning workflows"
$ git push origin main
```

Langkah 4: Pantau hasil workflow di GitHub

- Buka repository di GitHub
- Klik tab "Actions"

- Klik workflow "Container Security Scan" yang baru saja berjalan
- Lihat detail setiap step, terutama hasil Trivy scan
- Jika ada CVE CRITICAL/HIGH, pipeline akan gagal (exit-code: "1")

Langkah 5: Lihat hasil scan di GitHub Security tab

- Buka repository di GitHub
- Klik tab "Security" → "Code scanning"
- Hasil scan Trivy dalam format SARIF akan muncul di sini
- Setiap CVE ditampilkan dengan detail: lokasi, severity, dan rekomendasi fix

> **i INFO** GitHub Security tab hanya tersedia untuk repository public, atau repository private dengan GitHub Advanced Security.

4.3 Strategi Fail Fast vs Soft Fail

Terdapat dua pendekatan dalam mengintegrasikan scanning ke CI/CD:

Fail Fast (Direkomendasikan untuk Production):

- exit-code: "1" — pipeline langsung gagal jika ada CVE kritis
- Developer wajib fix vulnerability sebelum merge ke main
- Memastikan tidak ada kode vulnerable yang masuk ke production

Soft Fail (untuk proyek yang baru mulai):

- exit-code: "0" — pipeline tetap sukses meskipun ada CVE
- CVE tetap dilaporkan ke GitHub Security tab
- Tim punya waktu untuk memprioritaskan dan fix vulnerability

> ⚠ **PERHATIAN** Gunakan Soft Fail hanya untuk fase awal onboarding. Segera beralih ke Fail Fast setelah baseline keamanan sudah bersih.

> ✅ **CATATAN** Lab 4 selesai! Kamu sudah berhasil mengintegrasikan Trivy ke GitHub Actions untuk scanning otomatis setiap kali ada perubahan kode.

Tugas Praktikum

> **i INFO** Kerjakan tugas berikut dan dokumentasikan hasilnya dalam laporan praktikum (format PDF, maksimal 10 halaman).

Tugas Wajib (Bobot 60%)

Tugas 1 – Analisis Vulnerability (20%)

- Scan 3 image Docker yang berbeda: satu image versi lama dan satu image versi terbaru dari package yang sama, plus satu image dari microservice yang kamu pilih.
- Buat tabel perbandingan jumlah CVE berdasarkan severity (CRITICAL, HIGH, MEDIUM, LOW).
- Jelaskan mengapa versi terbaru selalu direkomendasikan dari sisi keamanan.

Tugas 2 – Secure Dockerfile (20%)

- Buat Dockerfile untuk aplikasi web sederhana pilihan kamu (Python Flask, Node.js Express, atau Go HTTP server).
- Terapkan minimal 5 best practices keamanan dari yang sudah dipelajari.
- Scan hasilnya dengan Trivy dan lampirkan screenshot hasilnya.

Tugas 3 – GitHub Actions Pipeline (20%)

- Buat GitHub repository dengan Dockerfile yang sudah aman.
- Implementasikan workflow GitHub Actions dengan Trivy scanning.
- Screenshot halaman Actions yang menunjukkan workflow berhasil/gagal.
- Lampirkan link repository GitHub kamu.

Tugas Pengayaan (Bobot 40%)

Tugas 4 – Falco Runtime Security (20%)

- Instal Falco menggunakan Docker (lihat referensi di bawah).
- Jalankan container dan lakukan aksi yang mencurigakan (misal: buka shell, baca /etc/passwd).
- Dokumentasikan alert yang dihasilkan Falco.

```
# Menjalankan Falco via Docker
$ docker run --rm -d --name falco \
    --privileged \
    -v /var/run/docker.sock:/host/var/run/docker.sock \
    -v /dev:/host/dev \
    -v /proc:/host/proc:ro \
    -v /boot:/host/boot:ro \
    -v /lib/modules:/host/lib/modules:ro \
    -v /usr:/host/usr:ro \
    falcosecurity/falco:latest

# Lihat alert Falco
$ docker logs -f falco
```

Tugas 5 – Container Hardening Checklist (20%)

- Gunakan checklist yang ada di slide perkuliahan.
- Terapkan minimal 10 item dari checklist ke Dockerfile kamu.
- Buat dokumen bukti penerapan (screenshot + penjelasan) untuk setiap item.

Format Laporan

Format	PDF, maksimal 10 halaman (tidak termasuk lampiran)
Nama file	Praktikum9_NIM_Nama.pdf
Konten	1) Judul & identitas, 2) Langkah pengerjaan per Lab, 3) Screenshot bukti, 4) Analisis & kesimpulan
Deadline	1 minggu setelah praktikum (sesuai jadwal dosen)
Pengumpulan	Upload ke LMS / link yang diberikan dosen

Referensi & Sumber Belajar

Dokumentasi Resmi

- Docker Documentation: https://docs.docker.com
- Trivy Documentation: https://aquasecurity.github.io/trivy
- GitHub Actions Documentation: https://docs.github.com/actions
- Falco Documentation: https://falco.org/docs

Standar & Best Practices

- OWASP Docker Security Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html
- CIS Docker Benchmark: https://www.cisecurity.org/benchmark/docker
- NIST Container Security Guide (SP 800-190): https://csrc.nist.gov/publications/detail/sp/800/190/final

Platform Latihan

- Docker Play: https://labs.play-with-docker.com (tidak perlu instal lokal)
- GitHub Skills: https://skills.github.com (tutorial GitHub Actions gratis)
- Katacoda / O'Reilly: https://learning.oreilly.com/scenarios

Video & Course

- TechWorld with Nana – Docker Crash Course: https://www.youtube.com/@TechWorldwithNana
- Trivy Tutorial by Aqua Security: https://www.youtube.com/@AquaSecurityOSS